

Explicación de `main.py` : mi primer agente con LangChain

Este documento explica, línea por línea, el archivo [main.py](#) — un agente mínimo construido con **LangChain**, un framework (conjunto de librerías) para construir aplicaciones sobre **LLM** (*Large Language Model*, modelo de lenguaje de gran tamaño, como GPT, Claude o Llama).

¿Qué es LangChain?

LangChain no reemplaza al modelo de lenguaje: es una **capa de abstracción** por encima de él. Su valor está en tres cosas:

1. **Interfaz común entre proveedores** — puedes cambiar de OpenAI a Anthropic a Ollama (modelos locales) cambiando un string, sin reescribir tu lógica.
2. **Tools** (herramientas) — funciones de código real que el modelo puede "pedir" ejecutar (por ejemplo, consultar una API, hacer una búsqueda, leer una base de datos).
3. **Orquestación del ciclo agente** — LangChain (usando **LangGraph**, su motor de grafos de ejecución) maneja el bucle: *el modelo decide usar una herramienta → la herramienta se ejecuta → el resultado regresa al modelo → el modelo responde*. Ese ciclo es, en esencia, lo que define a un **agente** de inteligencia artificial (IA): un sistema que no solo genera texto, sino que puede **actuar** sobre su entorno mediante herramientas y decidir sus propios siguientes pasos.

El código completo

```
# pip install -qU langchain langchain-ollama
from langchain.agents import create_agent

def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"

agent = create_agent(
    model="ollama:llama3.2:latest",
    tools=[get_weather],
    system_prompt="You are a helpful assistant",
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]}
```

```
)  
print(result["messages"][-1].content_blocks)
```

Explicación línea por línea

```
# pip install -qU langchain langchain-ollama
```

No es código, es un comentario a modo de recordatorio de instalación. Los dos paquetes que necesita este script:

- `langchain`: el core del framework — trae `create_agent` y la lógica general de agentes.
- `langchain-ollama`: el **adaptador** (paquete puente) específico para hablar con **Ollama**, la herramienta que sirve modelos de lenguaje de forma local en tu propia máquina (sin depender de una nube).

```
from langchain.agents import create_agent
```

Importa la función fábrica que construye un agente completo (modelo + herramientas + instrucciones) en una sola llamada. Sin esta función tendrías que armar tú mismo el grafo de ejecución con `LangGraph`, nodo por nodo.

```
def get_weather(city: str) -> str:
```

Define la **tool** (herramienta) que el agente podrá usar. Es una función Python normal, pero con dos detalles cruciales para que funcione como herramienta de un agente:

- **Type hints** (`city: str`, `-> str`): le indican al modelo qué tipo de dato espera cada parámetro. LangChain convierte esta firma en un esquema tipo JSON (formato estructurado de datos) que se envía al modelo.
- **Docstring** (`"""Get weather for a given city."""`): no es un comentario decorativo. Es la descripción que el modelo lee para decidir *cuándo* conviene usar esta herramienta. Si el docstring es vago, el modelo puede usarla mal o no usarla cuando debería.

```
    return f"It's always sunny in {city}!"
```

El cuerpo de la función. En este ejemplo es una simulación (siempre responde "soleado"); en un caso real, aquí llamarías a una **API** (*Application Programming Interface*, interfaz que permite que dos programas se comuniquen) de clima real.

```
agent = create_agent(...)
```

Construye el agente. Cada argumento:

Argumento	Qué hace
<code>model="ollama:llama3.2:latest"</code>	Indica qué modelo usar y con qué proveedor . El formato es <code>proveedor:nombre_modelo</code> . Aquí <code>ollama</code> es el proveedor (servidor local) y <code>llama3.2:latest</code> es el modelo específico. Sin el prefijo <code>ollama:</code> , LangChain no puede saber a qué proveedor mandar la petición y lanza un error.
<code>tools=[get_weather]</code>	La lista de funciones disponibles para el modelo. Podrían ser varias; aquí solo hay una.
<code>system_prompt="You are a helpful assistant"</code>	Instrucción base (<i>system prompt</i>) que condiciona el comportamiento del modelo durante toda la conversación — define su "rol".

El resultado (`agent`) es un **grafo ejecutable** compilado con LangGraph: una máquina de estados con nodos como "llamar al modelo" y "ejecutar herramienta", conectados por reglas que deciden el flujo según lo que el modelo pida.

```
result = agent.invoke(...)
```

Ejecuta el agente con una entrada. La entrada es un diccionario con la clave `"messages"`, siguiendo el formato estándar `role + content` (similar al usado por la mayoría de APIs de chat):

```
{"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]}
```

Al invocar, ocurre internamente este ciclo (el corazón de un agente):

1. El mensaje del usuario llega al modelo (`llama3.2`, corriendo en Ollama).
2. El modelo analiza la pregunta y decide que necesita el clima de San Francisco → genera una **tool call** (llamada a herramienta): `"ejecuta get_weather(city='San Francisco')"`.
3. LangGraph intercepta esa decisión, ejecuta tu función Python real, y agrega el resultado (`"It's always sunny in San Francisco!"`) al historial como un mensaje de tipo `tool`.
4. El historial actualizado (incluyendo el resultado de la herramienta) se envía de nuevo al modelo.
5. El modelo, ya con el dato real en mano, genera la respuesta final en lenguaje natural.

`result` es un diccionario con la clave `"messages"`: la lista completa de mensajes generados durante ese ciclo (mensaje del usuario → llamada a la herramienta → resultado de la herramienta → respuesta final del asistente).

```
print(result["messages"][-1].content_blocks)
```

- `result["messages"][-1]` toma el **último mensaje** de la lista: la respuesta final del asistente, ya generada después de usar la herramienta.
- `.content_blocks` imprime esa respuesta en su forma **estructurada** (una lista de bloques de contenido), en lugar de `.content`, que devolvería solo el texto plano.

Resumen mental

Piensa en el agente como:

Agente = LLM + Caja de herramientas + Bucle de decisión

`create_agent` te ahorra escribir ese bucle a mano con LangGraph. El "cerebro" que decide **si** llamar a `get_weather` y **con qué argumento** es el propio modelo (`llama3.2`) — tu código solo define qué herramientas existen y las ejecuta cuando el modelo las solicita.

Glosario

- **LangChain**: framework para construir aplicaciones sobre modelos de lenguaje, con soporte multi-proveedor y orquestación de agentes.
- **LangGraph**: motor de grafos de ejecución que usa LangChain internamente para manejar el flujo de un agente como una máquina de estados.
- **LLM** (*Large Language Model*): modelo de lenguaje de gran tamaño, entrenado para generar y entender texto (ej. Llama, GPT, Claude).
- **Ollama**: herramienta que permite correr LLMs de forma local en tu propia máquina, sin depender de una API en la nube.
- **Tool** (herramienta): función de código real que un agente puede invocar para actuar sobre su entorno (consultar una API, leer un archivo, hacer un cálculo, etc.).
- **System prompt**: instrucción inicial que define el rol o comportamiento base del modelo durante toda la conversación.
- **API** (*Application Programming Interface*): interfaz que permite que dos programas se comuniquen entre sí.